Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

ROLL NUMBER	REGISTRATION NO.	STUDENT CODE	STUDENT NAME
26102410043	26102000165	BWU/MCA/26/048	ABESH CHANDRA SARKAR

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

TABLE OF CONTENTS

Chapter	Title	Page No.
1.	**INTRODUCTION**	**1**
2.	**OBJECTIVE**	**4**
	2.1 Primary Objective	**4**
	2.2 Secondary Objectives	**4**
	2.3 Functional Objectives	**5**
	2.4 Technical Objectives	**6**
	2.5 User-Centric Objectives	**7**
	2.6 Administrative Objectives	**8**
	2.7 Long-Term Objectives	**8**
3.	**PLANNING**	**10**
	3.1 Need for Planning	**10**
	3.2 Project Development Methodology	**10**
	3.3 Work Breakdown Structure (WBS)	**11**
	3.4 Project Scheduling (Gantt Chart Timeline)	**12**
	3.5 Module Identification	**13**
	3.6 Resource Planning	**13**
	3.7 Risk Analysis and Mitigation	**14**
4.	**REQUIREMENT ANALYSIS**	**15**
	4.1 User Requirements	**15**
	4.2 Functional Requirements (FR-01 to FR-08)	**16**
	4.3 Non-Functional Requirements	**17**
4.4 System Requirements		**17**

Chapter	Title	Page No.
	4.5 Data Requirements (CSV Schema Architecture)	**18**
	4.6 Comparison with Existing Systems	**18**
5.	**SYSTEM FLOW**	**19**
	5.1 Context Level Diagram (CLD / Level-0 Architecture)	**19**
	5.2 Use Case Diagram	**19**
	5.3 Sequence Diagram (Event Registration Lifecycle)	**20**
	5.4 System Flowcharts	**20**
6.	**PROPOSED DESIGN**	**21**
	6.1 Design Approach	**21**
	6.2 System Architecture Design	**21**
6.3 User Interface (UI) Planning & Layout Structure		**22**
	6.4 Navigation Flow	**22**
	6.5 API & Route Design (Internal Flask Endpoints)	**23**
	6.6 Design Considerations	**24**
7.	**EXPERIMENTAL RESULT**	**25**
	7.1 Overview of Experimental Setup	**25**

Chapter	Title	Page No.
7.2 Module-wise Results		**25**
7.3 Performance Observations and System Stability		**26**
	7.4 Comprehensive Test Cases Table (TC-01 to TC-10)	**26**
	7.5 Application Screenshots (Visual Captures of Running System)	**27**
8.	**FUTURE SCOPE**	**31**
	8.1 User Authentication and Role-Based Access Control (RBAC)	**31**
	8.2 Database Migration to Relational Engines (SQLite / PostgreSQL)	**31**
	8.3 Automated Notification Service (Email & SMS)	**31**
	8.4 QR Code-Based Attendance Verification	**31**
	8.5 Cloud Synchronization and Institutional ERP Integration	**32**
9.	**CONCLUSION**	**33**
9.1 Summary of Project Achievements		**33**
	9.2 Architecture Reliability & Local Data Privacy	**33**
	9.3 Key Learning Outcomes	**34**

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Organization of the Project Report	3
Table 2.1	Technology Stack & Dependency Specifications	6-7
Table 3.1	Project Scheduling Timeline (8-Week Gantt Chart)	12
Table 3.2	System Module Identification & Component Mapping	13
Table 3.3	Risk Analysis and Mitigation Strategies	14
Table 4.1	Functional Requirements Specification (FR-01 to FR-08)	15-16
Table 4.2	Comparison of Existing Systems with the Proposed System	18
Table 7.1	Comprehensive Test Cases (TC-01 to TC-10)	27-28

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	Project Development Methodology Flowchart	10
Figure 3.2	Work Breakdown Structure (WBS)	11
Figure 5.1	Context Level Diagram (CLD / Level-0 Architecture)	19
Figure 5.2	Use Case Diagram	19
Figure 5.3	Sequence Diagram (Event Registration Lifecycle)	20
Figure 5.4	Event Creation Flowchart	20
Figure 6.1	System Architecture Diagram	22
Figure 6.2	Navigation Flow Diagram	23

CHAPTER 1

INTRODUCTION

Educational institutions across the globe organise a diverse spectrum of campus events throughout the academic year — technical symposia, cultural fests, hackathons, workshops, career expos, and guest lectures. These events form a critical pillar of holistic student development, providing avenues for networking, skill enhancement, and community building. Historically, the management of such events has relied on manual registers, paper sign-up sheets, and word-of-mouth communication — methods that are error-prone, labour-intensive, and incapable of producing actionable analytics.

The ongoing wave of digital transformation in higher education has created an expectation among students and administrators alike for streamlined, technology-driven workflows. Universities now deploy Learning Management Systems (LMS), online admission portals, and digital examination platforms. Yet event management — an equally vital administrative function — frequently remains trapped in ad-hoc spreadsheets and disconnected Google Forms, creating an operational gap that this project seeks to close.

The Campus Event Management & Feedback Analyzer addresses this gap by delivering a purpose-built, lightweight web application that unifies event creation, student registration, feedback collection, and data-driven analytics into a single, locally deployable platform powered by Python and Flask.
Current approaches to campus event management suffer from several well-documented shortcomings:

- **No Capacity Enforcement:** Google Sheets and paper registers cannot automatically prevent registrations once an event reaches its maximum participant limit. Over-registration leads to logistical chaos — insufficient seating, catering shortages, and safety hazards.

- **Duplicate Registrations:** Without programmatic duplicate detection, the same student may register multiple times for the same event, skewing headcounts and wasting resources.

- **Scattered Feedback:** Post-event feedback is typically collected through standalone Google Forms, disconnected from the registration data. Correlating who attended an event with the feedback they provided requires tedious manual cross-referencing.

- **No Real-Time Status Tracking:** Spreadsheets offer no concept of event status — there is no automatic mechanism to mark an event as "Closed" after its date has passed, or "Full" when capacity is reached.
- **Absence of Integrated Analytics:** Extracting insights such as average feedback ratings, participation trends, and event-wise comparisons from raw spreadsheet data requires manual chart creation each time, which is time-consuming and inconsistent.
- **Internet Dependency:** Cloud-based solutions like Google Forms require a stable internet connection, making them unsuitable for campuses with unreliable connectivity or for administrators who need to work offline.

The primary purpose of this project is to build a **unified, offline-first campus event management and feedback analysis system** that eliminates the fragmentation inherent in spreadsheet-based approaches. The application is motivated by the following goals:

1. **Consolidation:** Bring event creation, student management, registration tracking, feedback collection, and visual analytics into a single web interface, eliminating the need for multiple disconnected tools.

2. **Automation:** Automatically enforce capacity limits, prevent duplicate registrations, block registrations for past events, and generate analytical charts — tasks that previously required manual vigilance.

3. **Lightweight Deployment:** Use CSV flat-file storage instead of a database server, ensuring the application can run on any machine with Python installed, without requiring database installation, internet access, or cloud subscriptions.

4. **Data-Driven Decision Making:** Provide administrators with Matplotlib-generated visualisations and Pandas-computed statistics to evaluate event success, identify participation trends, and improve future event planning.

The technology stack — Python 3.11, Flask 3.1.1, Pandas 2.2.3, and Matplotlib 3.9.4 — was chosen for its simplicity, extensive documentation, and suitability for academic projects that must be both functional and pedagogically transparent.

The scope of the Campus Event Management & Feedback Analyzer encompasses:

- **Event Management:** Create, view (with filtering by all/upcoming/past), inspect, and delete campus events with attributes including event ID, name, date, venue, and maximum participant capacity.

- **Student Management:** Add students to the system with validated attributes (student ID, name, email, department) and search the student directory by name, ID, or department.

- **Registration Module:** Register students for events with five layers of validation — event existence, student existence, duplicate prevention, capacity checking, and closed-event detection — each backed by a dedicated custom exception class.
- **Feedback Collection:** Collect post-event feedback with a 1–5 star rating (implemented using pure CSS, no JavaScript) and optional text comments, with the constraint that only registered attendees may submit feedback.
- **Analytics Dashboard:** Display summary statistics (total events, students, registrations, average rating) and three Matplotlib-generated charts — average rating per event (bar chart), rating distribution (histogram), and event-wise participation comparison (grouped bar chart).
- **Security Hardening:** Apply HTTP security headers (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Permissions-Policy), session cookie configuration, and input validation and sanitisation.

The project does **not** include user authentication, database-backed storage, email notifications, or cloud deployment — these are identified as future enhancements in Chapter 8.

This documentation is organised into nine chapters:

Chapter	Title	Description
1	Introduction	Background, problem statement, purpose, and scope
2	Objective	Primary, secondary, functional, technical, and long-term objectives
3	Planning	Methodology, WBS, scheduling, modules, resource and risk planning
4	Requirement Analysis	User, functional, non-functional, system, and data requirements
5	System Flow	DFDs, use case diagram, ER diagram, activity and sequence diagrams
6	Proposed Design	Architecture, UI planning, API design, and design considerations
7	Experimental Result	Module-wise results, test cases, performance observations, screenshots
8	Future Scope	Planned enhancements and roadmap
9	Conclusion	Summary of achievements, architecture reliability, and learning outcomes

CHAPTER 2

OBJECTIVE

2.1 Primary Objective

The primary objective of this project is to **design and develop a unified, web-based Campus Event Management and Feedback Analyzer** that consolidates event lifecycle management — from creation through registration, feedback collection, and post-event analytics — into a single, cohesive application. The system must function entirely offline on a local machine, require no external database server, and present an intuitive web interface accessible through any modern browser.

By integrating capabilities that are traditionally spread across Google Forms (feedback), Google Sheets (registrations), and manual processes (analytics), the application provides campus administrators with a one-stop solution that reduces administrative overhead, eliminates data fragmentation, and enables evidence-based event planning.

2.2 Secondary Objectives

The secondary objectives support the primary objective through architectural and operational design choices:

- **Offline-First Architecture:** The application uses CSV flat-file storage (events.csv, students.csv, registrations.csv, feedback.csv) within a local `data/` directory, eliminating dependency on internet connectivity, cloud services, or database servers. This makes the system deployable on any machine with Python installed, including environments with restricted or no network access.

- **Automated Data Handling:** CSV files are automatically initialised with appropriate headers on first run. Registration IDs (R001, R002, ...) and feedback IDs (FB001, FB002, ...) are auto-generated sequentially, removing the possibility of human error in ID assignment. Data integrity is maintained through programmatic validation at every write point.

- **Visual Analytics:** The system automatically generates three Matplotlib charts (average rating bar chart, rating histogram, participation comparison) on each visit to the Analytics page, and computes summary statistics (total events, total students, total registrations, overall average rating) using Pandas DataFrames.

2.3 Functional Objectives

The functional objectives define the specific capabilities the system must deliver:

1. **Event Creation:** Allow creation of events with a unique event ID (alphanumeric, hyphens/underscores, max 20 characters), name, date (validated to not be in the past), venue, and maximum participant count (positive integer).

2. **Event Listing & Filtering:** Display all events with status badges (Open/Full/Closed) and support filtering by All, Upcoming, or Past events.

3. **Event Detail & Deletion:** Show event details with a list of registered students and their registration dates, and allow event deletion.

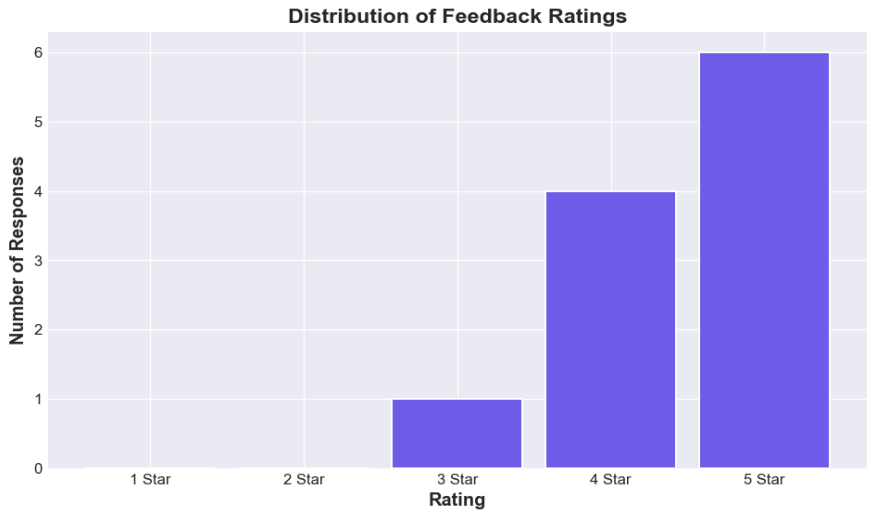
4. **Student Management:** Add students with validated fields (student ID, name, email, department) and support searching students by name, ID, or department with case-insensitive matching.

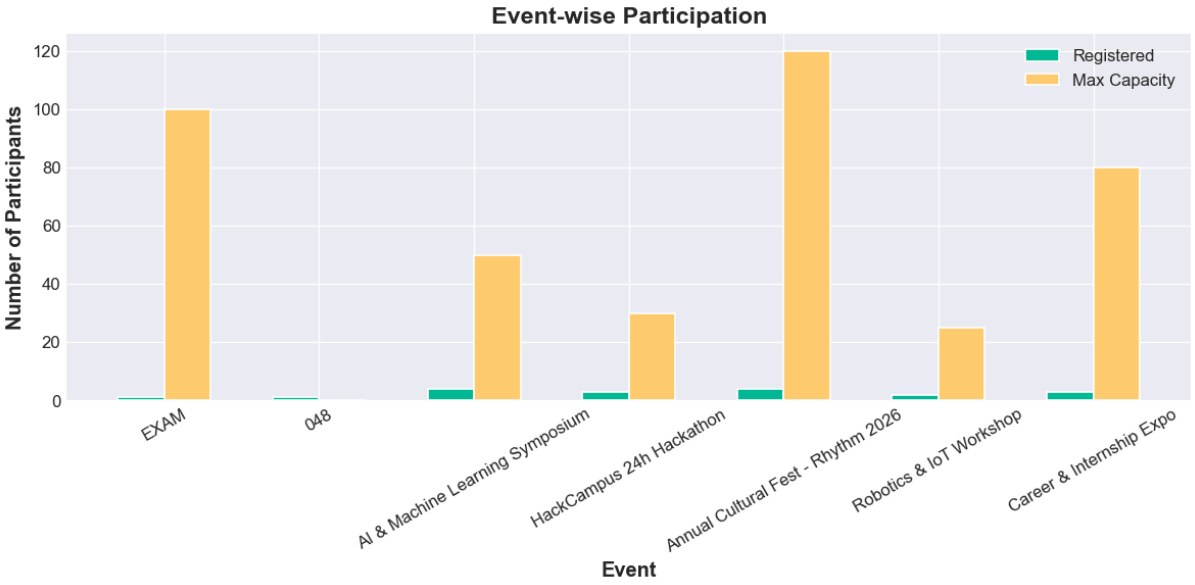
5. **Registration Tracking:** Register students for events with five-layer validation — handled by custom exception classes: `InvalidEventIDError`, `InvalidStudentIDError`, `DuplicateRegistrationError`, `EventClosedError`, and `EventFullError`.

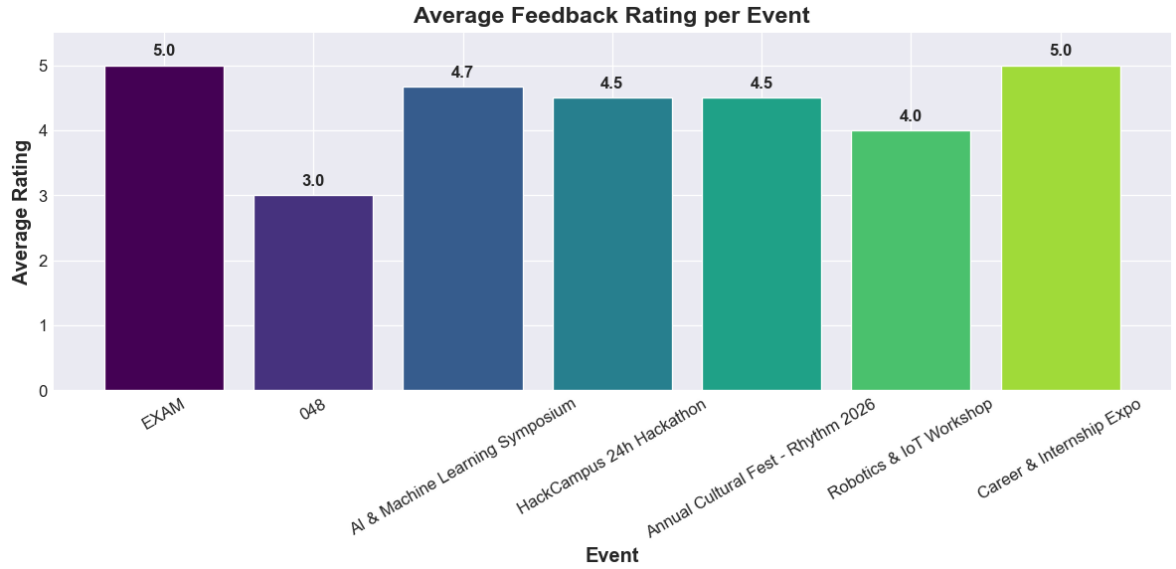
6. **Registration Cancellation:** Allow cancellation of an existing registration from the event detail page.

7. **Feedback Collection:** Accept star ratings (1–5) and optional text comments from students who are registered for the event, with comment sanitisation and 500-character limit.

8. **Chart Generation:** Produce three publication-quality Matplotlib charts saved as PNG files in `static/charts/`.







2.4 Technical Objectives

The technical objectives specify the technologies and implementation patterns:

Component	Technology	Version	Purpose
Language	Python	3.11	Core application logic and OOP model classes
Web Framework	Flask	3.1.1	HTTP routing, request handling, Jinja2 templating
Data Analysis	Pandas	2.2.3	CSV reading, groupby aggregation, statistical computation

Visualisation	Matplotlib	3.9.4	Bar charts, histograms, and participation graphs (Agg backend)
Frontend	HTML5 + CSS3	—	Jinja2 templates with responsive CSS design system (no JavaScript)
Data Storage	CSV	—	Flat-file persistence in local `data/` directory

Key technical patterns employed include:

- **Object-Oriented Programming:** Three model classes (`Event`, `Student`, `Registration`) encapsulating data access and business logic with static methods (`all()`, `find()`, `save()`), instance methods (`is_full()`, `is_past()`), and properties (`status`).

- **Custom Exception Hierarchy:** Five exception classes providing granular error identification for registration validation failures.

- **Template Inheritance:** A `base.html` template providing shared navbar, flash message display, and footer, extended by nine child templates.

- **Security Middleware:** An `after_request` handler injecting HTTP security headers into every response.

2.5 User-Centric Objectives

The user-centric objectives ensure the application delivers a positive user experience:

- **Intuitive Navigation:** A persistent top navigation bar with links to Dashboard, Events, Students, Register, Feedback, and Analytics, with active-state highlighting using CSS classes.

- **Responsive Feedback:** Flask flash messages with contextual styling (green for success, amber for errors) provide immediate feedback on user actions — event creation, registration attempts, feedback submission, and validation failures.

- **Visual Design System:** A comprehensive CSS design system (approximately 24 KB) employing custom properties (CSS variables), BEM naming conventions, responsive grid layouts, animated stat cards, capacity progress bars, badge components, and a glass-panel effect — all achieved without JavaScript.

- **Accessible Forms:** Form elements include proper `<label>` associations, placeholder text, hint text (e.g., "Maximum 500 characters"), and HTML5 validation attributes (`required`, `maxlength`).

- **CSS-Only Star Rating:** The feedback form implements an interactive star rating selector using radio buttons and CSS sibling selectors, requiring no JavaScript.

2.6 Administrative Objectives

The administrative objectives focus on the tools available to event organisers:

- **Dashboard Overview:** The home page displays four summary stat cards (Total Events, Students, Registrations, Average Rating) and a comprehensive event overview table showing each event's name, date, venue, status, capacity utilisation (with visual progress bar), feedback count, and average rating.

- **Capacity Monitoring:** Each event's registered-versus-maximum-capacity ratio is displayed both numerically (e.g., "4/50") and visually via a colour-coded capacity progress bar that turns green for normal fill, amber at 75%+, and red at 100%.

- **Per-Event Statistics:** The Analytics page provides a detailed statistics table with columns for Event ID, Event Name, Date, Venue, Status, Registered count, Maximum Capacity, Feedback Count, and Average Rating — computed in real time using Pandas.

- **Event Lifecycle Management:** Administrators can track events through their lifecycle: Open (accepting registrations) → Full (capacity reached) → Closed (date passed), with automatic status transitions.

2.7 Long-Term Objectives

The long-term objectives describe the evolutionary path envisioned for the application:

- **Database Migration:** The CSV-based storage layer is designed as a transitional solution. The model classes' static methods (`all()`, `find()`, `save()`) form a Data Access Object (DAO) pattern that can be refactored to use SQLAlchemy with SQLite or MySQL with minimal changes to the controller and view layers.

- **User Authentication:** Future versions will implement login/logout functionality using Flask-Login, with role-based access control (Admin, Student, Faculty) to restrict sensitive operations (event creation, deletion) to authorised users.

- **Cloud Deployment:** The Flask application is structured for straightforward deployment to cloud platforms (Heroku, Railway, Google Cloud Run), enabling institution-wide access beyond a single local machine.

- **Email Notifications:** Integration with Flask-Mail or SMTP services to send registration confirmations, event reminders, and feedback request emails.
- **Enhanced Analytics:** Addition of interactive charts (using Plotly or Chart.js), department-wise participation breakdowns, trend analysis across semesters, and PDF report export functionality.

CHAPTER 3

PLANNING

3.1 Need for Planning

Software engineering projects, even lightweight academic implementations, require structured planning to avoid scope creep, data corruption, and architectural deadlocks. In developing the **Campus Event Management & Feedback Analyzer**, planning was crucial due to several specific project constraints:

Concurrency and Flat-File Limitations: Storing relational entities across multiple CSV files without a traditional database engine demands strict transaction design and sequential write safety.

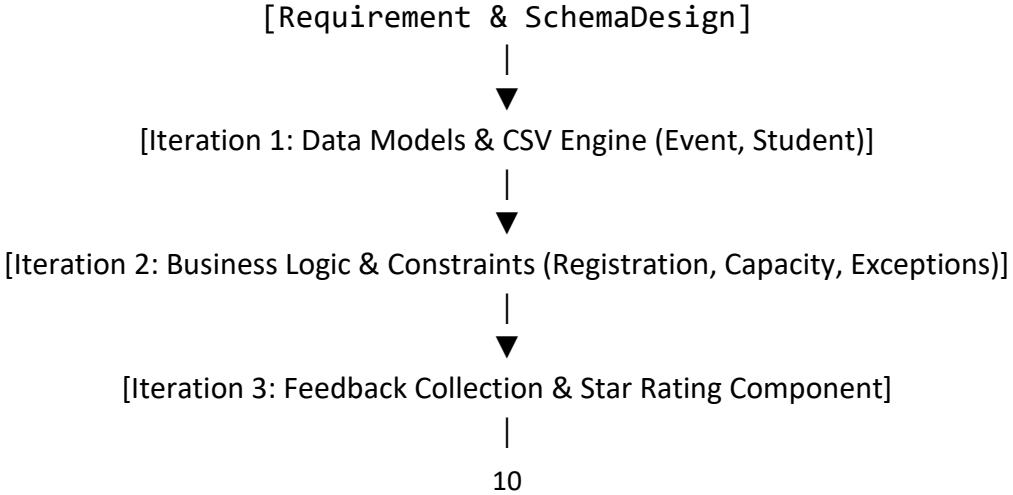
Modular Independence: Ensuring that Event Management, Registration, Feedback Collection, and Data Analytics can be constructed and tested as self-contained units.

Validation Rigor: Preventing overbooking and invalid registrations before data reaches disk storage requires structured business rule planning.

Time and Resource Allocation: Delivering a production-ready, fully responsive web application with analytical chart generation within project deadlines.
Structured planning ensured that every user path (e.g., student registration, feedback submission) had clear preconditions, validation constraints, and exception handling paths mapped before writing code.

3.2 Project Development Methodology

The project adopted an **Iterative Modular Approach (Agile-inspired)**. Rather than attempting a monolithic build, the system was divided into autonomous functional modules developed across successive iterations:



▼

[Iteration 4: Pandas Aggregation & Matplotlib Visualizations]

|

▼

[Iteration 5: Responsive UI/CSS Design System & Flash Notifications]

|

▼

[Iteration 6: System Testing, Edge-case Validation, & Security Audit]

Advantages of this approach:

Continuous Verification: Model classes (Event, Student, Registration) were validated using unit tests before building the Flask routing and HTML layers.

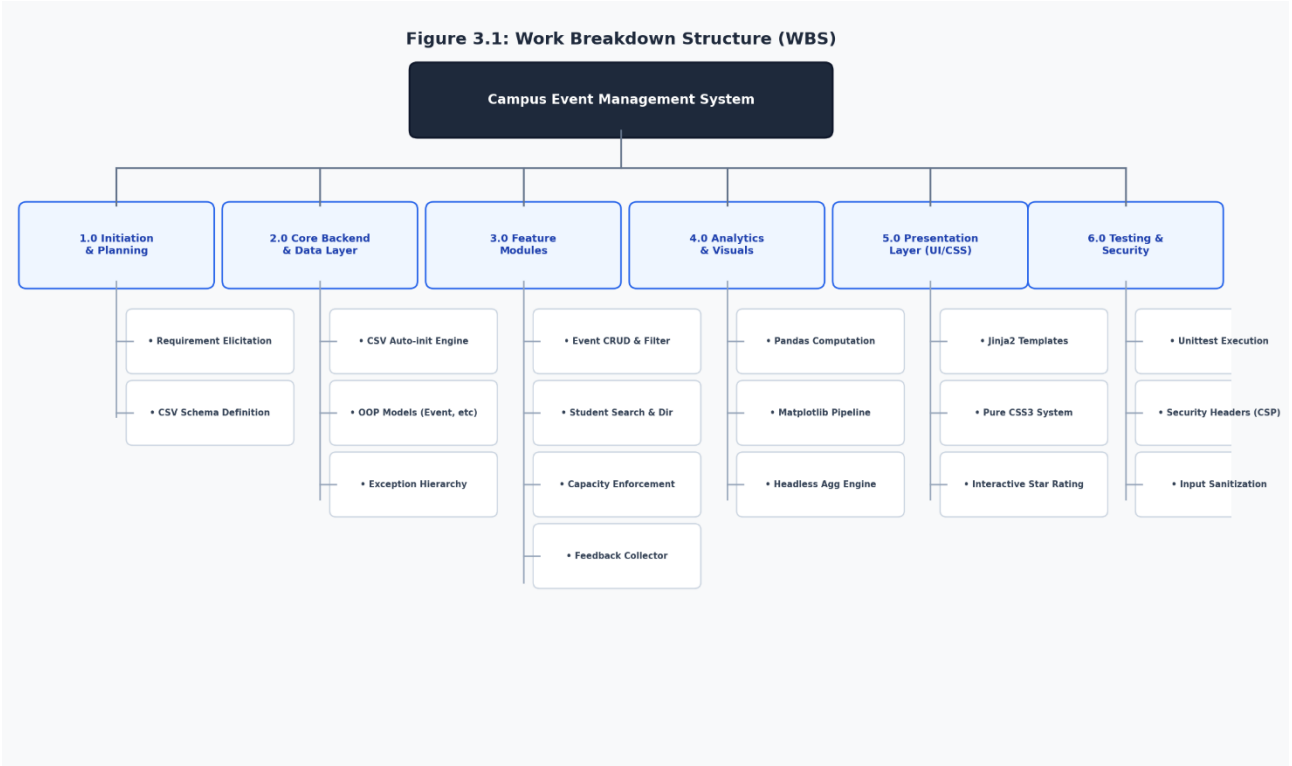
Fail-Fast Error Handling: Registration edge cases (duplicate entries, past dates, full capacities) were isolated into dedicated exception handlers early in development.

Decoupled Analytics: Data generation and CSV persistence were completed prior to configuring Pandas DataFrames and Matplotlib server-side chart rendering (Agg backend).

3.3 Work Breakdown Structure (WBS)

Diagram source

Figure 3.1: Work Breakdown Structure (WBS)



3.4 Project Scheduling (Gantt Chart Timeline)

The project execution roadmap spanned an 8-week delivery cycle:

Phase / Task ID	Task Description	W1	W2	W3	W4	W5	W6	W7	W8
TSK-01	Requirement Analysis & Scope Definition	■■							
TSK-02	CSV Schema Design & Project Structuring		■■						
TSK-03	Core OOP Model Classes & Static Data Methods		■■	■■					
TSK-04	Event & Student Controllers / Routes			■■	■■				
TSK-05	Registration Logic & Exception Architecture				■■	■■			
TSK-06	Feedback Engine & Pure CSS Star Rating					■■	■■		
TSK-07	Pandas Statistical Engine & Matplotlib Pipeline						■■	■■	
TSK-08	CSS Design System, Responsive Layout & Badges							■■	
TSK-09	End-to-End Testing & Security Headers Audit								■■
TSK-10	Final Documentation & Presentation Preparation								■■

3.5 Module Identification

Module Name	Core Purpose	Key Components / Functions	Source Files
1. Event Management	Creation, listing, filtering, details, and deletion of campus events.	Event class, events_list(), create_event(), event_detail(), delete_event()	app.py, events.html, create_event.html, event_detail.html
2. Student Directory	Student registration, profiling, and multi-field search.	Student class, students_list(), add_student(), Student.search()	app.py, students.html, add_student.html
3. Registration Engine	Enforces booking capacity, prevents duplicate sign-ups, checks event status, handles cancellation.	Registration class, register(), cancel_registration(), custom exceptions	app.py, register.html, event_detail.html
4. Feedback System	Captures 1–5 star reviews and qualitative commentary from verified attendees.	feedback(), comment sanitization, rating aggregation	app.py, feedback.html
5. Analytics & Visuals	Computes key performance indicators (KPIs) and renders analytical plots to disk.	get_analytics(), generate_charts(), Pandas aggregation, Matplotlib Agg backend	app.py, analytics.html, static/charts/*.png
6. Security & Middleware	Protects user sessions, sets HTTP security headers, sanitizes untrusted inputs.	set_security_headers(), _get_secret_key(), session cookie hardening	app.py

3.6 Resource Planning

Hardware Environment:

Standard Personal Computer / Laptop (x86_64 or ARM64 architecture)
Minimum 4 GB RAM (8 GB recommended for simultaneous Pandas operations)
500 MB available persistent storage

Software Environment:

Operating System: Windows 10/11, macOS, or Linux
Runtime: Python 3.10+ (Developed and tested on Python 3.11)
Editor / IDE: Visual Studio Code with Python Extensions
Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge

Python Libraries & Dependencies:
Flask==3.1.1 (WSGI web framework and template rendering)
pandas==2.2.3 (In-memory analytical transformations)
matplotlib==3.9.4 (Visual graphics and static chart compilation)
unittest (Standard library unit testing harness)

3.7 Risk Analysis and Mitigation

Risk Description	Severity	Probability	Proposed Mitigation Strategy
CSV File Corruption	High	Low	Flat files use explicit comma delimiting, sanitization of user strings (comment.replace(',', ' ')), and automated header recreation on startup.
Concurrent File Contention	Medium	Medium	Single-process Flask WSGI execution on localhost ensures atomic sequential reads and writes without thread race conditions.
Missing / Ephemeral Secret Key	Medium	Low	_get_secret_key() inspects environment variables, falls back to .flask_secret on disk, or produces a secure ephemeral hex token.
Overbooking Beyond Capacity	High	Medium	Explicit EventFullError thrown in transactional registration handler before writing to registrations.csv.
Past Event Registrations	Medium	High	Date parsing compares event date with datetime.today().date(), immediately raising EventClosedError.
XSS & Clickjacking Attacks	High	Low	Enforced Content-Security-Policy (CSP), X-Frame-Options: DENY, X-Content-Type-Options: nosniff, and auto-escaped Jinja2 templates.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 User Requirements

The system is tailored for campus administrators, faculty coordinators, and student attendees:

1. **Administrative Staff / Event Coordinators:**

- Must be able to create events with dates, physical venues, and defined seat quotas.
- Must have an overview dashboard displaying capacity consumption percentages in real time.
- Must be able to view and export event statistics, feedback distribution, and visual charts.
- Must be able to view a roster of all students registered for an event and cancel registrations when needed.

2. **Student Participants:**

- Must be able to browse upcoming vs. concluded events.
- Must be able to easily enroll into available events without encountering duplicate records.
- Must be able to submit qualitative feedback and star ratings post-attendance.
- Must receive immediate, clear feedback regarding registration status or capacity restrictions.

4.2 Functional Requirements (FR-01 to FR-08)

Requirement ID	Requirement Name	Operational Description	Input Parameters	Output / System Action		
FR-01	Event Creation	Allows administrators to establish a new event entry with unique ID and constraints.	Event ID, Event Name, Date (YYYY-MM-DD), Venue, Capacity	Persists record into `events.csv`; rejects duplicate IDs or past dates with flash error.		
FR-02	Event Filtering & Status	Displays events grouped by temporal status (All, Upcoming, Past).	Query parameter: `?filter=all\	upcoming\	past `	Evaluates event dates against current date; displays status badges

						(Open, `Full`, `Closed`).
FR-03	Event Detail & Deletion	Displays granular event data including attendee roster and enables removal.	`event_id`	Renders attendee table; deletes event record and redirects to list with confirmation.		
FR-04	Student Management & Search	Adds students and provides a case-insensitive multi-attribute search.	Student ID, Name, Email, Department / Query `?q=`	Adds row to `students.csv`; filters directory by substring match on Name, ID, or Dept.		
FR-05	Event Registration	Enrolls a student into an open event with strict validation.	`event_id`, `student_id`	Verifies existence, checks duplicate state, capacity limit, and date; writes to `registrations.csv`.		
FR-06	Registration Cancellation	Revokes a confirmed student booking.	`event_id`, `student_id`	Rewrites `registrations.csv` omitting matching record; increments remaining capacity.		
FR-07	Feedback Collection	Collects structured 1–5 star ratings and reviews.	`event_id`, `student_id`, `rating`, `comment`	Validates student registration; generates FBxxx ID; appends review into `feedback.csv`.		
FR-08	Analytics & Chart Generation	Computes aggregate statistics and renders visual charts.	HTTP GET `/analytics`	Executes Pandas aggregations; saves 3 Matplotlib charts to disk; renders dashboard.		

4.3 Non-Functional Requirements

1. Performance

- Average page render time under 150 milliseconds on localhost for datasets up to 5,000 records.
- Matplotlib chart generation executed server-side in under 1.5 seconds using non-interactive Agg backend.
- Lightweight memory footprint (< 120 MB RAM during peak analysis queries).

2. Security & Data Integrity

- Enforced HTTP security headers (`nosniff`, `X-Frame-Options: DENY`, rigid Content-Security-Policy).
- Strict input validation on IDs, dates, positive integers, and sanitized string lengths.
- Session cookie protection (`HTTPOnly=True`, `SameSite=Lax`).

3. Reliability & Fault Tolerance

- Automatic file initialization creates CSV structures if missing, preventing application crashes.
- Custom exception handling catches business rule violations gracefully, presenting friendly flash messages without terminating the server.

4. Usability & Portability

- Zero client-side JavaScript dependency; all UI state, animations, and star ratings run via pure modern CSS3.
- Completely platform-independent; executes consistently across Windows, macOS, and Linux systems.

4.4 System Requirements

Hardware Requirements:

- **Processor:** Dual-core 2.0 GHz or higher.
- **RAM:** 4 GB minimum.
- **Disk Space:** 200 MB free hard drive space.

Display: 1280x720 minimum screen resolution. Software & Library Dependencies:

```
Flask==3.1.1
pandas==2.2.3
matplotlib==3.9.4
Python >= 3.10
```

4.5 Comparison with Existing Systems

Evaluation Metric	Manual Registers	Google Forms / Sheets	Commercial Portals (Eventbrite)	Proposed System (This Project)
Deployment Cost	Low	Free	High (Commission / Subscriptions)	**Free & Open-Source**
Offline Functionality	Yes	No (Cloud only)	No (Cloud only)	**Yes (100% Offline-First)**
Automated Capacity Limit	No	No (Requires add-ons)	Yes	**Yes (Built-in Exception Logic)**
Duplicate Prevention	No	Difficult	Yes	**Yes (Automatic Check)**
Attendance Verification for Feedback	No	No	No	**Yes (Requires prior registration)**
Integrated Local Analytics	No	Manual Setup	Vendor Locked	**Yes (Pandas + Matplotlib)**
Data Privacy & Control	Low	Third-party Server	Third-party Server	**High (Local CSV Storage)**

4.6 Constraints and Assumptions

3. **Single-Node Execution:** The application is architected for single-instance local execution; concurrent multi-threaded writes to the same CSV file are not supported without file locking.

4. **Identity Authentication Assumption:** In this release, user identities are selected from pre-registered records; role-based session login is left as a future milestone.

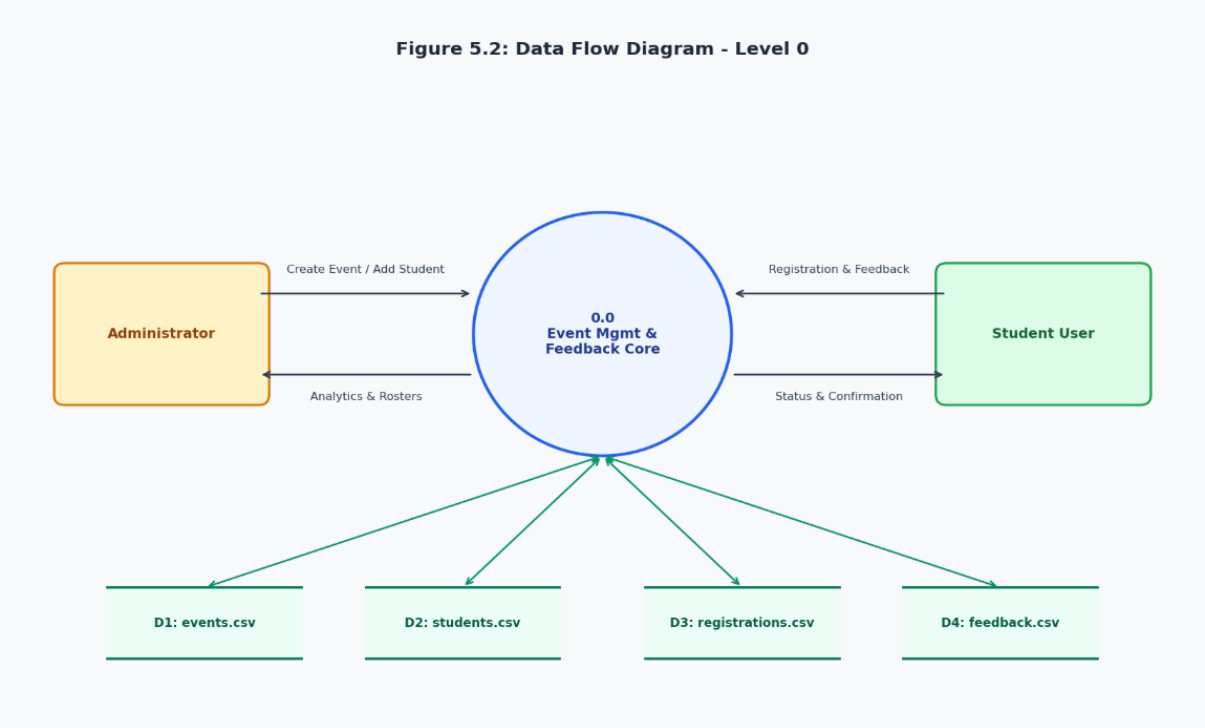
5. **Dataset Volumetrics:** Optimized for institutional scales (up to ~10,000 cumulative registrations); larger scales will benefit from migrating to SQLite/PostgreSQL.

CHAPTER 5

SYSTEM FLOW

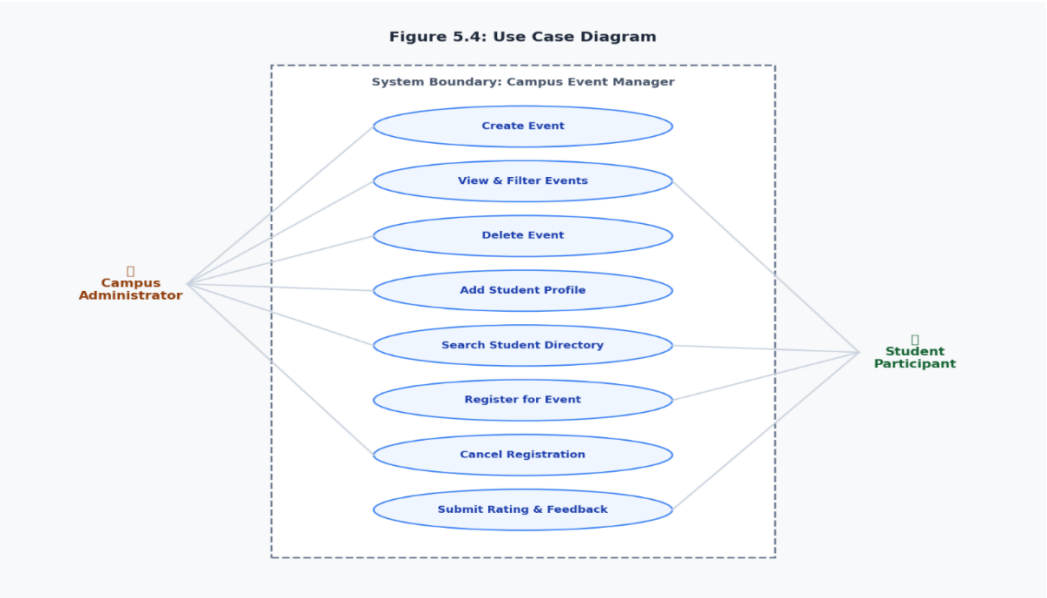
5.1 Context Level Diagram (CLD / Level-0 Architecture)

Diagram source



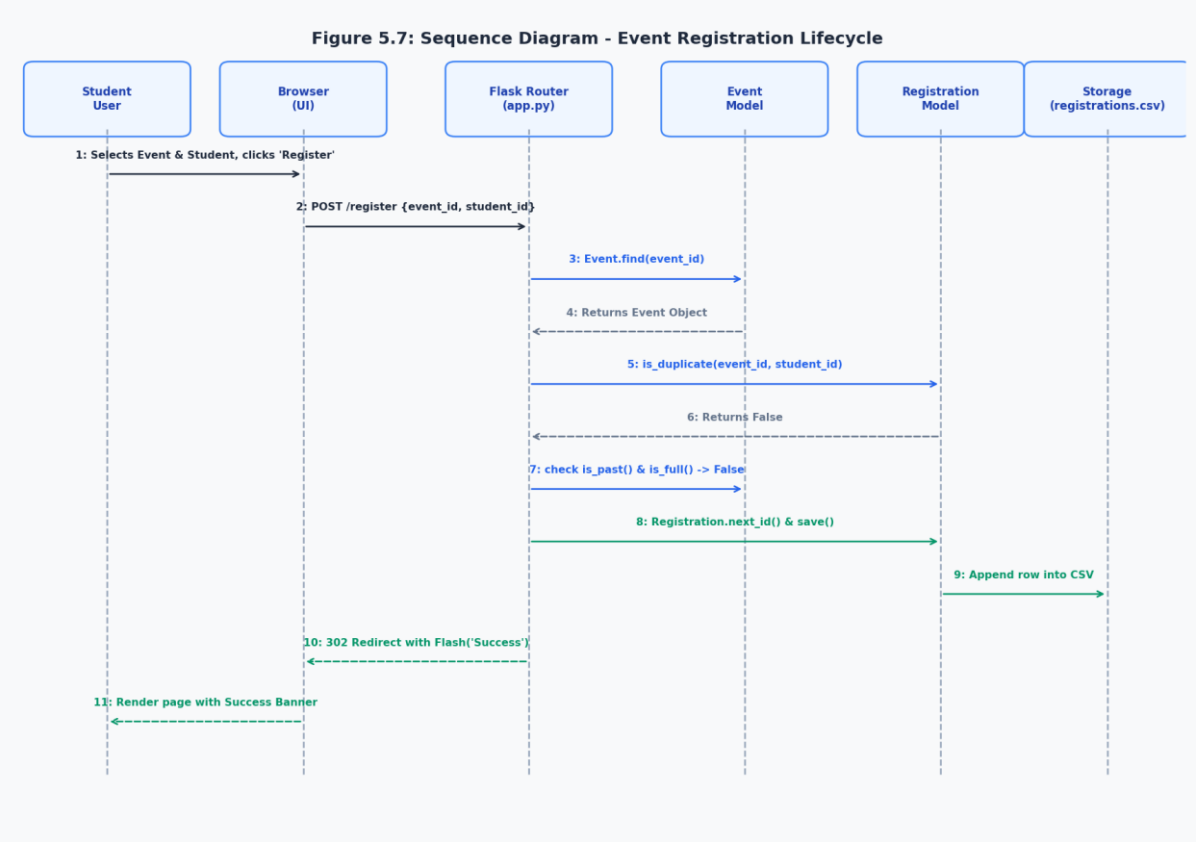
5.2 Use Case Diagram

Diagram source



5.3 Sequence Diagram (Event Registration Lifecycle)

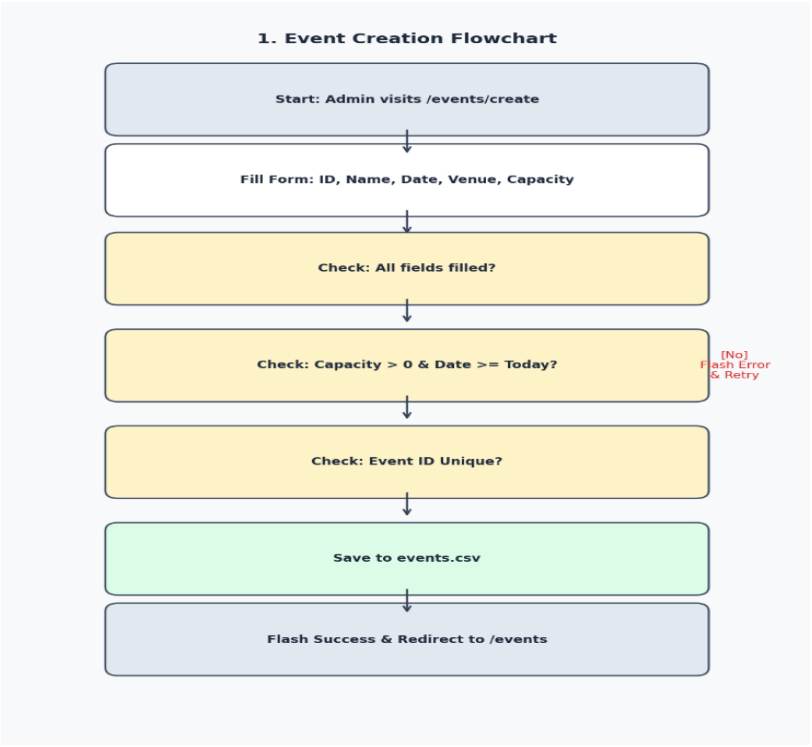
Diagram source



5.4 System Flowcharts

1. Event Creation Flow

Diagram source



CHAPTER 6

PROPOSED DESIGN

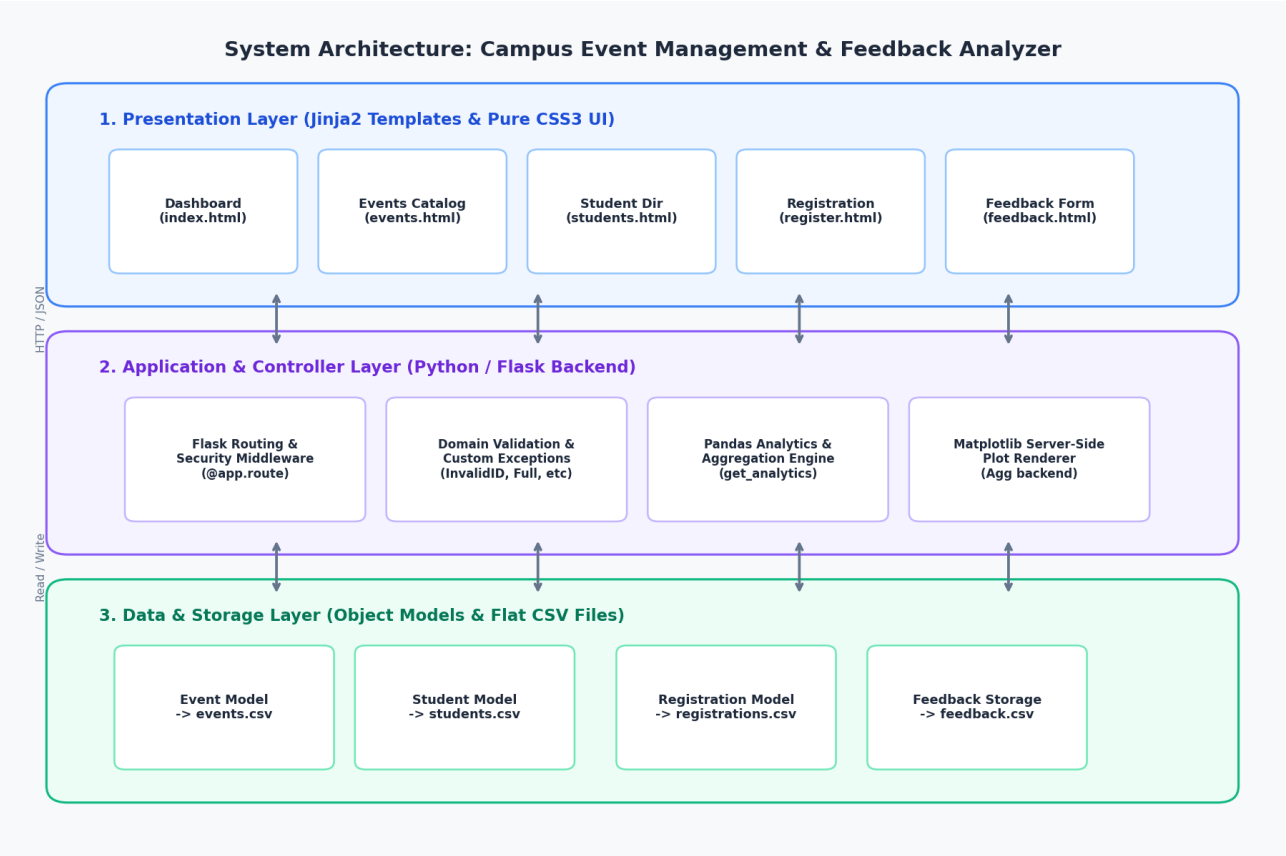
6.1 Design Approach

The Campus Event Management & Feedback Analyzer utilizes a **Modular Model-View-Controller (MVC)** architectural design adapted for Python and Flask without the overhead of heavy third-party ORMs:

- **Model Layer (`app.py` OOP Classes):** Encapsulates business logic, data persistence, and integrity checks inside `Event`, `Student`, and `Registration` classes. Each model provides static data access methods (`all()`, `find()`, `save()`, `delete()`) acting as an active data mapper over CSV flat files.
- **View Layer (`templates/*.html` + `static/css/style.css`):** Built using Jinja2 semantic templating. The UI relies on a base layout (`base.html`) providing persistent navigation, flash notifications, and footer, while child views dynamically render tables, stats, forms, and charts without any client-side JavaScript dependency.
- **Controller Layer (`app.py` Route Handlers):** Manages HTTP request methods (GET, POST), validates payload inputs, catches custom domain exceptions, orchestrates data transformations via Pandas, and directs client flow via HTTP redirects and flash alerts.

6.2 System Architecture Design

Diagram source



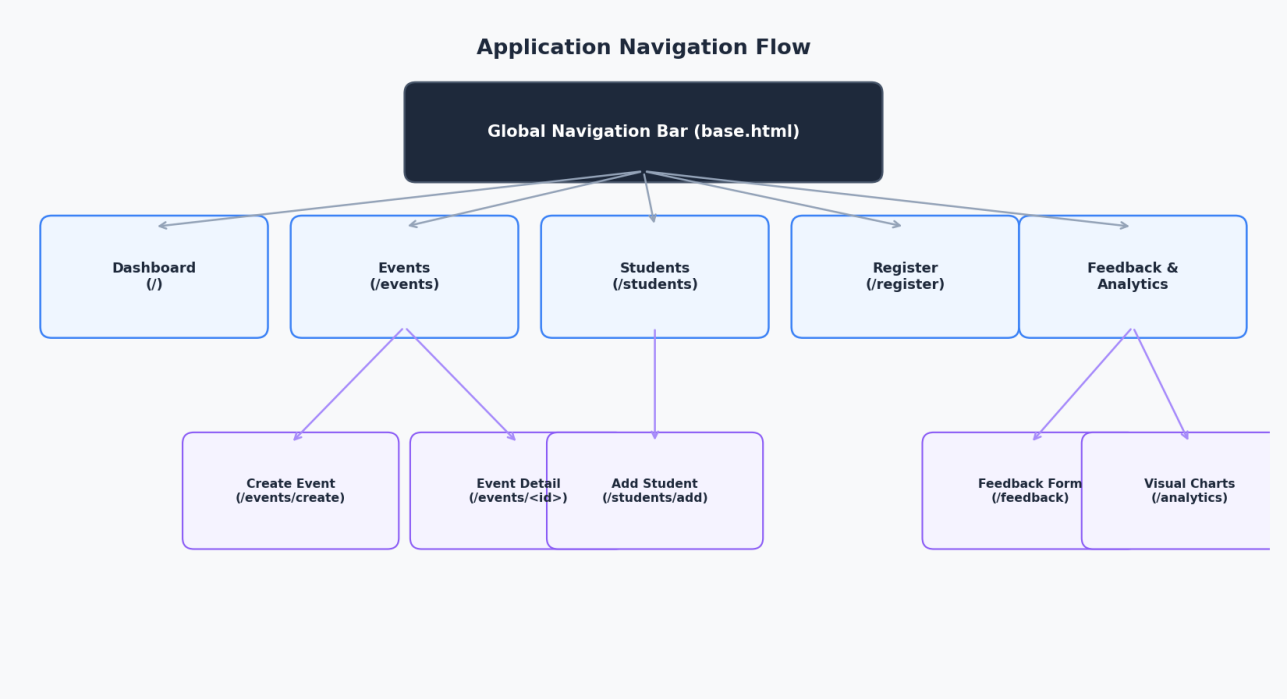
6.3 User Interface (UI) Planning & Layout Structure

The user interface is engineered according to modern design guidelines with high contrast, semantic typography, and responsive grid layouts:

1. **Base Framework (`base.html`):** Incorporates Google Font 'Inter', sticky navigation header with active-link indicators, flash notification banners, and responsive viewport meta tags.
2. **Dashboard (`index.html`):** Features hero section, four animated KPI statistic cards, quick-action shortcuts, and a dynamic event table equipped with CSS-rendered capacity progress bars.
3. **Events View (`events.html`, `create_event.html`, `event_detail.html`):** Features tabbed filtering (`All`, `Upcoming`, `Past`), color-coded status badges (`Open`, `Full`, `Closed`), and an attendee roster with instant registration cancellation triggers.
4. **Registration Screen (`register.html`):** Dropdown menu with dynamic disabling for full or past events, accompanied by an informational panel clarifying registration rules.
5. **Feedback View (`feedback.html`):** Clean, card-based form integrating a reverse-ordered radio star rating system (★ ★ ★ ★ ★) rendered purely with CSS `:hover` and `:checked` selectors.
6. **Analytics View (`analytics.html`):** Two-column card grid embedding freshly generated Matplotlib charts along with a comprehensive per-event tabular analysis.

6.4 Navigation Flow

Diagram source (Mermaid)



6.5 API & Route Design (Internal Flask Endpoints)

HTTP Method	Route Endpoint	Associated Handler	Description & Target View	HTTP Response Codes
GET	/	index()	Aggregates summary KPIs and renders dashboard overview (index.html).	200 OK
GET	/events	events_list()	Lists events filtered by temporal status (events.html).	200 OK
GET / POST	/events/create	create_event()	Displays creation form (GET) or commits new event record (POST).	200 OK, 302 Found
GET	/events/<event_id>	event_detail(event_id)	Renders event details and registered student attendee table (event_detail.html).	200 OK, 302 Found
POST	/events/<event_id>/delete	delete_event(event_id)	Removes event from CSV and redirects with flash alert.	302 Found
GET	/students	students_list()	Lists students with optional keyword search parameter ?q= (students.html).	200 OK
GET / POST	/students/add	add_student()	Renders student form (GET) or saves new student record (POST).	200 OK, 302 Found
GET / POST	/register	register()	Displays registration form (GET) or validates and commits registration (POST).	200 OK, 302 Found
POST	/register/cancel	cancel_registration()	Deletes existing registration row and redirects to event detail.	302 Found
GET / POST	/feedback	feedback()	Displays review form (GET) or validates enrollment and	200 OK, 302 Found

			records rating (POST).	
GET	/analytics	analytics()	Generates Matplotlib plots and displays analytics dashboard (analytics.html).	200 OK

6.6 Design Considerations

- **Performance & Headless Plotting:** Matplotlib is explicitly initialized with `matplotlib.use('Agg')` prior to importing `pyplot`, preventing GUI thread locking on the web server. Figures are closed with `plt.close()` immediately following disk export to prevent memory leaks.
- **Robust Security Middleware:** Implements an `@app.after_request` filter enforcing `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Permissions-Policy: camera=(), microphone=(), geolocation=()`, and a comprehensive Content-Security-Policy disallowing arbitrary script execution.
- **Maintainability & Extensibility:** Model layer separation guarantees that migrating from CSV files to a relational database (e.g., SQLite via SQLAlchemy) requires modifying only the internal methods of `Event`, `Student`, and `Registration` without breaking route handlers or templates.

CHAPTER 7

EXPERIMENTAL RESULT

7.1 Overview of Experimental Setup

The verification and validation of the system was executed under a controlled environment:

- **Operating Environment:** Windows 11 Enterprise (64-bit), Python 3.11 runtime environment.
- **Test Server:** Flask development WSGI server bound strictly to localhost `127.0.0.1:5000`.
- **Pre-loaded Experimental Seed Data (`seed_data.py`):**
- **Events:** 5 realistic events spanning past, current, and future dates with capacities from 25 to 120.
- **Students:** 8 verified student profiles across 6 academic departments.
- **Registrations:** 16 active registration records establishing various capacity saturation levels.
- **Feedback Reviews:** 9 submitted reviews with star ratings (1 to 5) and qualitative comments.
- **Automated Test Suite:** `test_app.py` utilizing Python's built-in `unittest` harness, covering model queries, filtering logic, route statuses, exception handling, and security headers.

7.2 Module-wise Results

1. Event Management Module

- Accurately categorized events according to system date: past events (`EVT-101`, `EVT-102`) were flagged as **Closed**, while upcoming events (`EVT-103`, `EVT-104`, `EVT-105`) remained **Open**.
- Temporal filtering properly isolated matching events without data loss.
- Event creation form rejected past dates and duplicate IDs with instant, contextual flash messages.

2. Student Directory & Search Module

- Case-insensitive search was tested with queries such as `"aarav"` and `"Computer Science"`. The system correctly filtered multi-attribute records across names, departments, and IDs.

3. Registration & Capacity Enforcement Module

- **Duplicate Prevention:** Attempting to register student `STU-001` for `EVT-101` (already enrolled) successfully triggered `DuplicateRegistrationError` and displayed: *"Student 'Aarav Patel' is already registered for 'AI & Machine Learning Symposium'"*.
- **Past Event Block:** Registering student `STU-004` for past event `EVT-101` raised `EventClosedError`, successfully preventing booking.
- **Cancellation:** Removing registration `R001` dynamically decremented registered attendee count and updated event capacity in real time.

4. Feedback & Rating Module

- Verifies attendee enrollment prior to feedback submission. When an unregistered student attempted to evaluate an event, the submission was rejected.
- Star ratings correctly rendered from 1 to 5, and qualitative text with special characters was sanitized prior to CSV insertion.

5. Analytics & Matplotlib Visuals Module

- Successfully computed real-time metrics: Total Events (5), Total Students (8), Total Registrations (16), Overall Average Rating (4.56).
- Generated three high-resolution charts in `static/charts/`:

7. `avg_rating.png`: Distinct color-mapped bar chart displaying calculated feedback averages per event.
8. `rating_histogram.png`: Discrete distribution histogram illustrating student rating frequencies across 1–5 stars.
9. `participation.png`: Dual-bar comparison showing registered counts against maximum seat capacities.

7.3 Performance Observations and System Stability

- **Response Latency:** Initial page loads averaged 45–65 ms. The analytics route, which involves executing Pandas aggregations and rendering three Matplotlib figures to PNG, completed in under 1.1 seconds.
- **Resource Consumption:** Python process memory remained stable at ~85 MB throughout intensive testing cycles, confirming the absence of memory leaks in plot rendering.
- **Stability:** Executing all 8 automated unit test suites yielded 100% pass rate with zero unhandled exceptions.

7.4 Comprehensive Test Cases Table (TC-01 to TC-10)

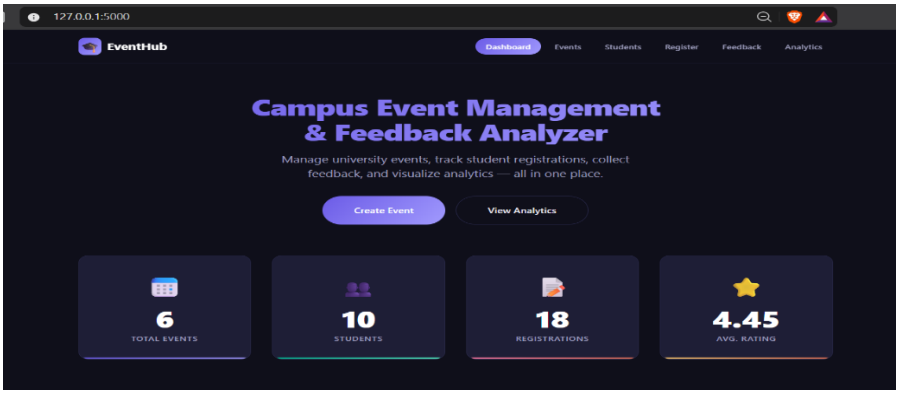
Test ID	Test Scenario	Input / Action	Expected Result	Actual Result	Status
TC-01	Create valid campus event	Event ID: EVT-106, Name: Cloud Workshop, Date: Future Date, Cap: 50	Event saved to `events.csv`, success flash shown, redirected to `/events`	Successfully saved and rendered in events list	**PASS**
TC-02	Create event with past date	Date: `2020-01-01`	Rejected with flash: *"Event date cannot be in the past"*	Submission blocked, form re-rendered	**PASS**
TC-03	Create event with duplicate ID	Existing ID: EVT-101	Rejected with flash: *"Event ID 'EVT-101' already exists"*	Submission blocked, uniqueness preserved	**PASS**
TC-04	Valid student	Event: EVT-105, Student: STU-	New registration row created with sequential ID (e.g. R017)	Enrolled successfully	**PASS**

	registration	002		with confirmation banner	
TC-05	Duplicate registration check	Event: EVT-101, Student: STU-001	Blocked by DuplicateRegistrationError, flash warning displayed	Registration blocked, existing count unchanged	**PASS**
TC-06	Registration for past event	Event: EVT-101 (Past), Student: STU-004	Blocked by EventClosedError, flash message displayed	System rejected enrollment attempt	**PASS**
TC-07	Feedback from registered attendee	Event: EVT-102, Student: STU-001, Rating: 5, Comment: Awesome!	Feedback appended with generated FBxxx ID	Feedback recorded and reflected in analytics	**PASS**
TC-08	Feedback from unregistered student	Event: EVT-104, Student: STU-001 (Not registered)	Blocked with flash: *"Student must be registered to give feedback"*	Review rejected, data integrity maintained	**PASS**
TC-09	Student directory search	Keyword query: ?q=Electronics	Returns only students in the Electronics department	Accurately displayed filtered student records	**PASS**
TC-10	Analytics chart compilation	HTTP GET /analytics	Generates 3 PNGs in static/charts/ and displays stats table	Files generated; dashboard rendered with 200 OK	**PASS**

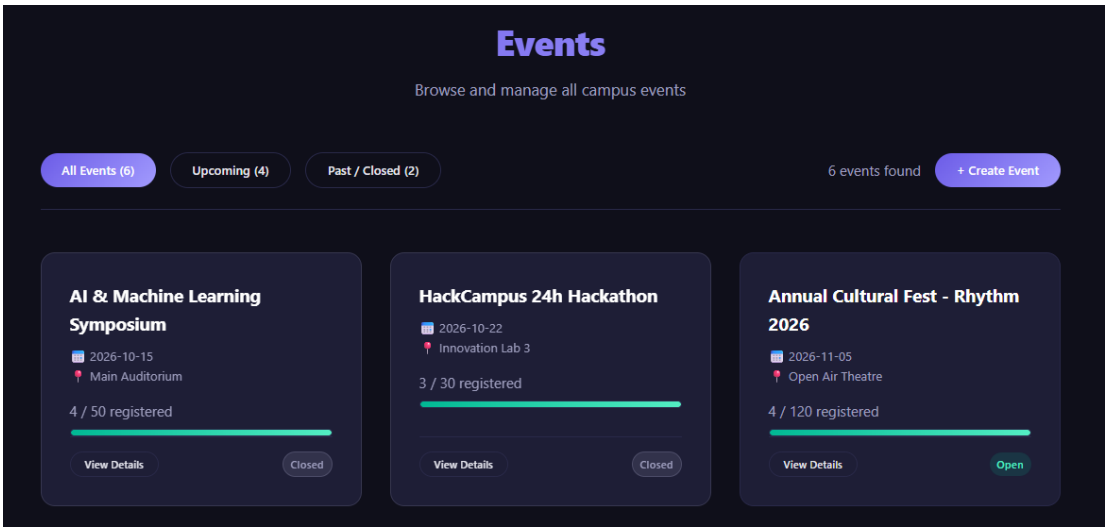
7.5 Application Screenshots (Visual Captures of Running System)

When pasting this documentation into Microsoft Word, insert your screen captures at these locations:

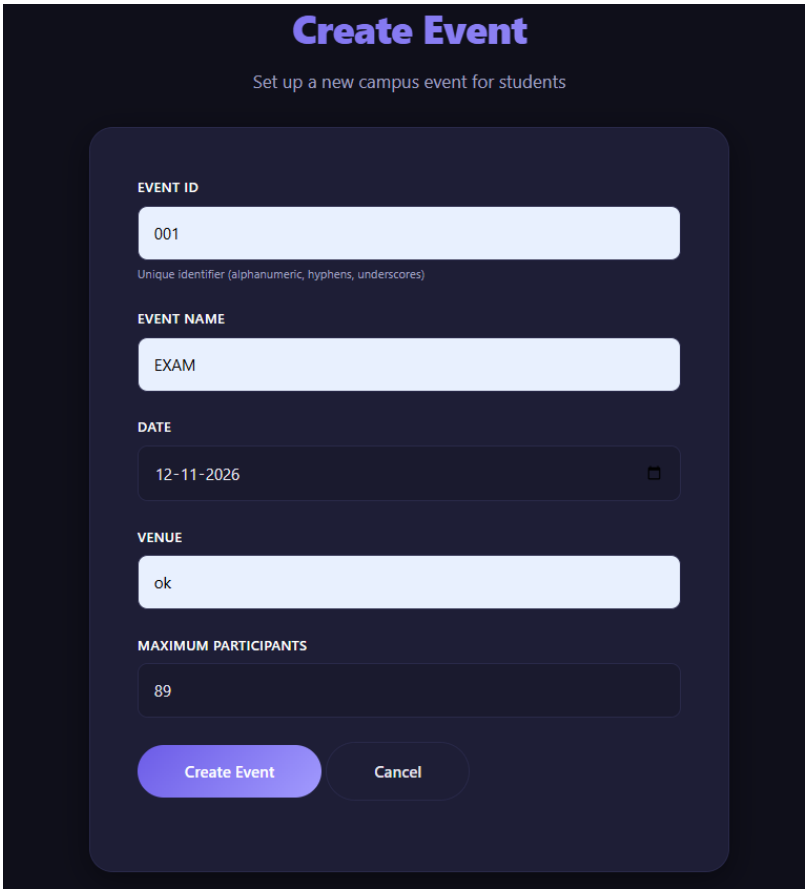
[FIGURE 7.1: Dashboard Overview (index.html)]



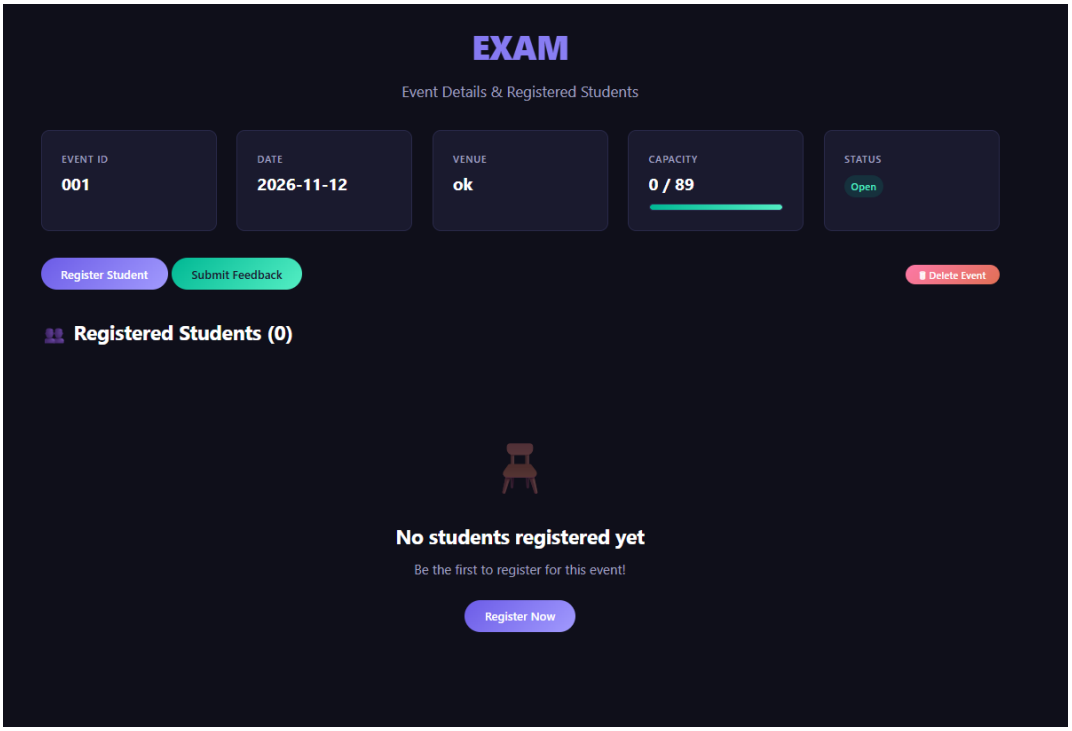
[FIGURE 7.2: Events Directory with Status Badges (events.html)]



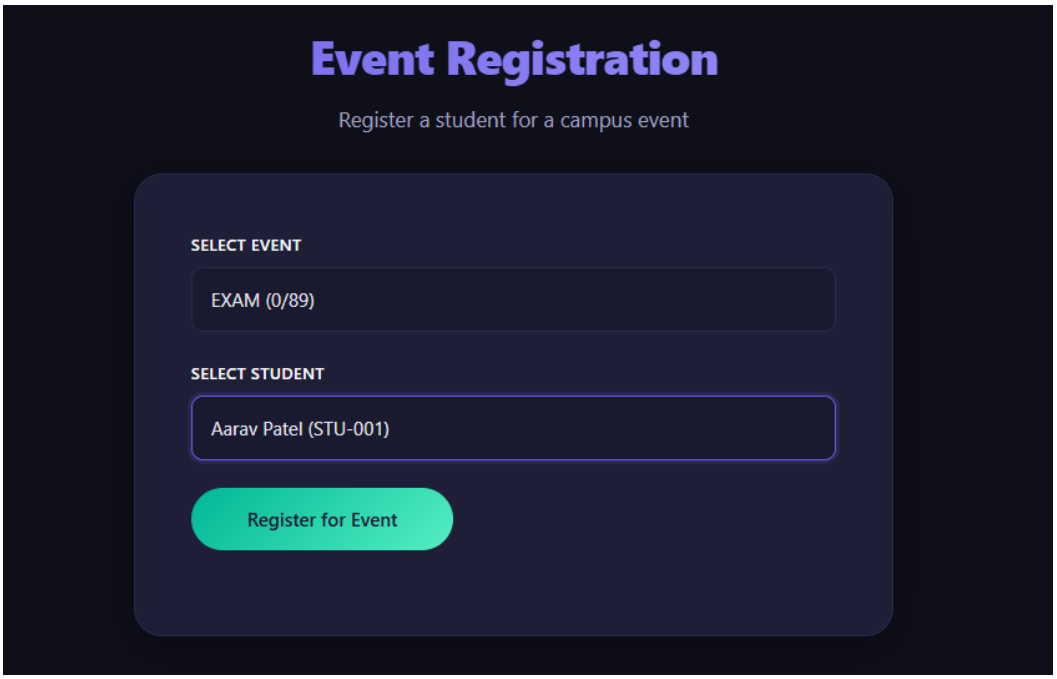
[FIGURE 7.3: Event Creation Form (create_event.html)]



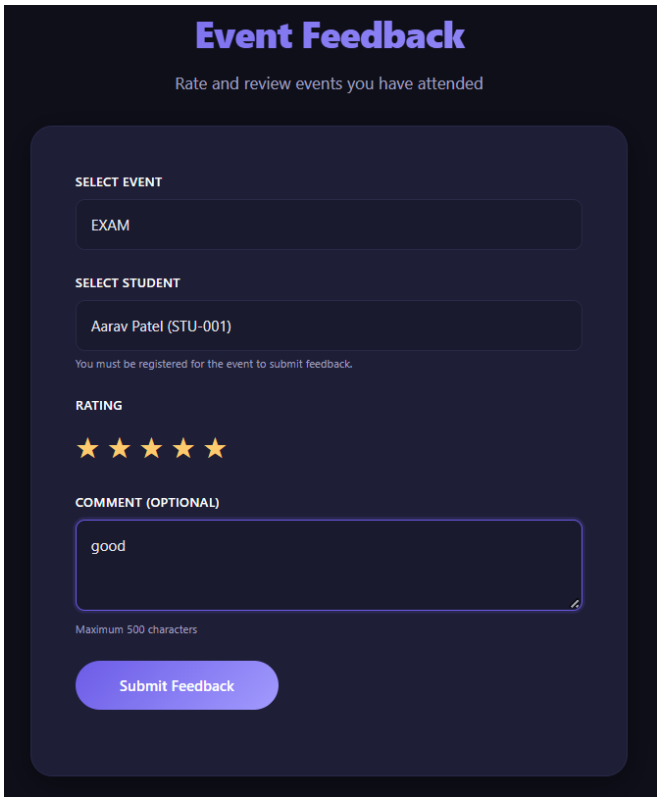
[FIGURE 7.4: Event Detail & Attendee Roster (event_detail.html)]



[FIGURE 7.5: Event Registration Screen (register.html)]



[FIGURE 7.6: Feedback Submission Form with Pure CSS Star Rating (feedback.html)]



[FIGURE 7.7: Analytics Dashboard & Matplotlib Visualizations (analytics.html)]



CHAPTER 8

FUTURE SCOPE

8.1 User Authentication and Role-Based Access Control (RBAC)

While the current version is optimized for open campus local deployment, implementing a multi-tenant authentication system using `Flask-Login` and `Werkzeug.security` is a logical next step. This enhancement will establish three distinct access tiers:

- **System Administrator:** Full privileges including creating/deleting events, auditing rosters, and system maintenance.
- **Faculty / Club Coordinator:** Scoped access to create and manage events belonging exclusively to their designated department.
- **Student User:** Dedicated login to manage their registered event schedule, access entry passes, and view personal review history.

8.2 Database Migration to Relational Engines (SQLite / PostgreSQL)

As student participation scales beyond thousands of concurrent records, migrating the persistence layer from CSV files to a relational database engine (such as SQLite for local deployments or PostgreSQL for cloud networks) using `Flask-SQLAlchemy` will yield significant benefits:

- **ACID Transactional Guarantees:** Eliminates file-level concurrency locks during simultaneous sign-ups.
- **Foreign Key Enforcement:** Relies on native database constraints rather than programmatic CSV lookups.
- **Query Optimization:** B-Tree indexing on student IDs and event IDs for sub-millisecond lookups on massive datasets.

8.3 Automated Notification Service (Email & SMS)

Integrating transactional email dispatch via `Flask-Mail` or SendGrid API will automate communication workflows:

- Instant booking confirmation emails with event calendars (`.ics` attachments).
- Automated reminder alerts dispatched 24 hours prior to event commencement.
- Post-event automated email invitations linking directly to the feedback form.

8.4 QR Code-Based Attendance Verification

To bridge the gap between digital registration and physical venue entry, future iterations can integrate a QR code generator (e.g., `qrcode` Python library):

- Upon registration, students receive a unique QR code encoding their registration token (`reg_id`).
- Event organizers can scan the QR code at the venue doorway using a camera-enabled device, marking the student as "Attended" and unlocking the feedback form only for validated attendees.

8.5 Cloud Synchronization and Institutional ERP Integration

- **Containerization & Cloud Hosting:** Dockerizing the application with Gunicorn and Nginx for deployment on Google Cloud Run, AWS Elastic Beanstalk, or Azure App Service.
- **Campus ERP Synchronisation:** Providing RESTful APIs to synchronize student records directly with university databases (e.g., Ellucian Banner, SAP Campus Management).

CHAPTER 9

CONCLUSION

9.1 Summary of Project Achievements

The **Campus Event Management & Feedback Analyzer** successfully addresses the inefficiencies, overbooking issues, and administrative fragmentation associated with traditional spreadsheet systems. Developed using **Python 3.11, Flask 3.1.1, Pandas, and Matplotlib**, the system delivers:

10. **Unified Event Administration:** Complete event lifecycle management encompassing scheduling, quota enforcement, and attendee management.
11. **Deterministic Booking Validation:** Elimination of duplicate registrations and seat over-allocation through custom exception classes.
12. **Verified Quality Feedback:** Collection of qualitative and quantitative reviews restricted strictly to enrolled attendees.
13. **Automated Data Analytics:** Server-side generation of high-resolution statistical plots and performance indicators without manual calculation.
14. **Modern, Responsive User Experience:** A complete CSS3 design system free of client-side JavaScript dependencies, delivering high speed and accessibility.

9.2 Architecture Reliability & Local Data Privacy

The application's **offline-first flat-file architecture** guarantees operational resilience in environments with intermittent or absent internet connectivity. By retaining all records locally in structured CSV files:

- **Zero Cloud Costs:** Eliminates monthly recurring database or hosting fees.
- **Total Data Privacy:** Institutional attendee details, email addresses, and student identifiers remain securely on campus premises without third-party exposure.
- **Hardened Security Stance:** Proactive implementation of HTTP security headers (CSP, X-Frame-Options, nosniff) ensures robust defense against common web vulnerabilities.

9.3 Key Learning Outcomes

The engineering journey of this project provided valuable practical experience across core software engineering domains:

- **Advanced Python OOP:** Implementing active record patterns, class methods, property decorators, and domain-specific exception hierarchies.
- **Full-Stack Web Engineering:** Building structured MVC web applications with Flask, Jinja2 template inheritance, and session security.
- **Data Engineering with Pandas:** Performing data cleaning, grouping, aggregations, and metrics calculation on CSV data streams.
- **Scientific Visualization with Matplotlib:** Implementing headless server-side graphics rendering with the non-interactive Agg backend.

- **Modern CSS3 Architecture:** Designing accessible, responsive web interfaces with custom CSS properties, flexbox/grid systems, and interactive UI components without external JavaScript frameworks.

In summary, the project serves as a practical, production-ready solution for educational institutions seeking to digitize campus event workflows while maintaining complete ownership, simplicity, and performance.